

UCS503P · SOFTWARE ENGINEERING PROJECT

Poolit

Pooled Quick-Commerce for Hostels

Project Report

Dev Ariwala — 1024170433

Paryag Kakkar — 1024170447

Odd Semester 2026–27

September 16, 2026

1. Abstract

Poolit is a pooled quick-commerce application built for college hostels. Rather than each student placing an individual delivery order and paying a full delivery fee, Poolit groups orders placed by students in the same hostel within a shared time-boxed window, called a slot. As more students join a slot, the per-student delivery fee drops in real time along a fixed ladder — ₹20 for the first one or two orders, falling to ₹10, then ₹5, and finally free once ten or more students have joined. This live-dropping fee counter is the product's central hook: it gives students an immediate, visible incentive to order within the same window as their hostel-mates.

The system consists of two React/TypeScript single-page applications — a mobile-first student app and a desktop vendor console — sharing a common domain package for types, business rules, and API access. An Express.js REST API backed by Supabase (PostgreSQL) enforces the fee ladder, slot and order lifecycles, and atomic inventory management, with a background scheduler auto-closing slots on timer expiry. The project was built iteratively over a semester: an in-memory frontend prototype validated the user experience before any backend existed, followed by a first backend on MongoDB, a migration to Supabase for relational integrity and built-in auth, the addition of vendor authentication, a vendor console redesign, deployment to Vercel, and a hardening pass addressing concurrency bugs and query performance. This report documents the problem, requirements, system design, development process, and outcomes of that work.

2. Introduction & Problem Statement

Quick-commerce and food delivery apps price delivery per order, regardless of how many nearby customers are ordering at the same time. In a college hostel, this is a structural inefficiency: dozens of students in the same building may order from the same vendor within minutes of each other, yet each pays a full, individual delivery fee, and the vendor dispatches many small deliveries to the same address instead of one consolidated trip.

Hostel students ordering food individually each pay a full delivery fee, discouraging frequent ordering and creating inefficiency for vendors doing many small deliveries. Poolit incentivizes students to order within the same time window so deliveries can be consolidated, cutting cost per student and delivery trips per vendor. The project targets two actors: students, who use a mobile-first app to browse, order, and track deliveries; and vendors, who use a desktop admin console to run pools across all hostels they serve, dispatch orders, and manage inventory.

3. Literature Review & Existing Solutions

Poolit's model sits at the intersection of two existing categories of app, neither of which solves the hostel delivery-fee problem directly.

Existing Solution	Delivery Fee Model	Gap for Hostel Use
Standard food delivery (Zomato, Swiggy)	Flat or distance-based fee charged per individual order	No mechanism to group orders from the same building into one delivery
Quick-commerce (Blinkit, Zepto, Instamart)	Flat fee, sometimes waived above an order-value threshold	Any batching happens internally at the dark store; the customer sees no incentive tied to neighbors ordering too
Campus / hostel ordering apps	Single campus storefront, usually fixed pickup slots	Fee reduction, where it exists, is not tied to real-time participation
Group-buying / social commerce (Pinduoduo-style)	Unit price drops as more buyers join a group order	Built for e-commerce SKUs over multi-day windows, not same-day hostel food delivery

None of these expose a live, per-window delivery fee tied specifically to co-located hostel residents. Poolit's contribution is combining the group-buying discount mechanic with the immediacy of quick-commerce, scoped to a fixed, naturally co-located population — a single hostel.

4. Requirements

4.1 Functional Requirements

- Students can select their hostel and browse that hostel's vendor menu.
- Students can add items to a cart and submit an order into the hostel's currently open pool.
- The system computes and displays the per-student delivery fee live, based on the number of distinct orders in the open slot.
- Students can track an order through Pooling → Preparing → On the way → Delivered.
- Students can view order history and reorder a previous order.
- Students can manage a profile, wallet, and delivery addresses.
- Vendors authenticate with email and password before accessing the console.
- Vendors can view every hostel's pools on a single dashboard.
- Vendors can close a pool early, or allow it to auto-close on timer expiry.
- Vendors can view a consolidated pick list per pool and dispatch closed pools.
- Vendors can mark dispatched orders as delivered.
- Vendors can restock inventory via presets or a custom amount, and receive low-stock alerts.
- Vendors can view analytics: revenue trend, orders by hour, a peak-hours heatmap, and a hostel breakdown.

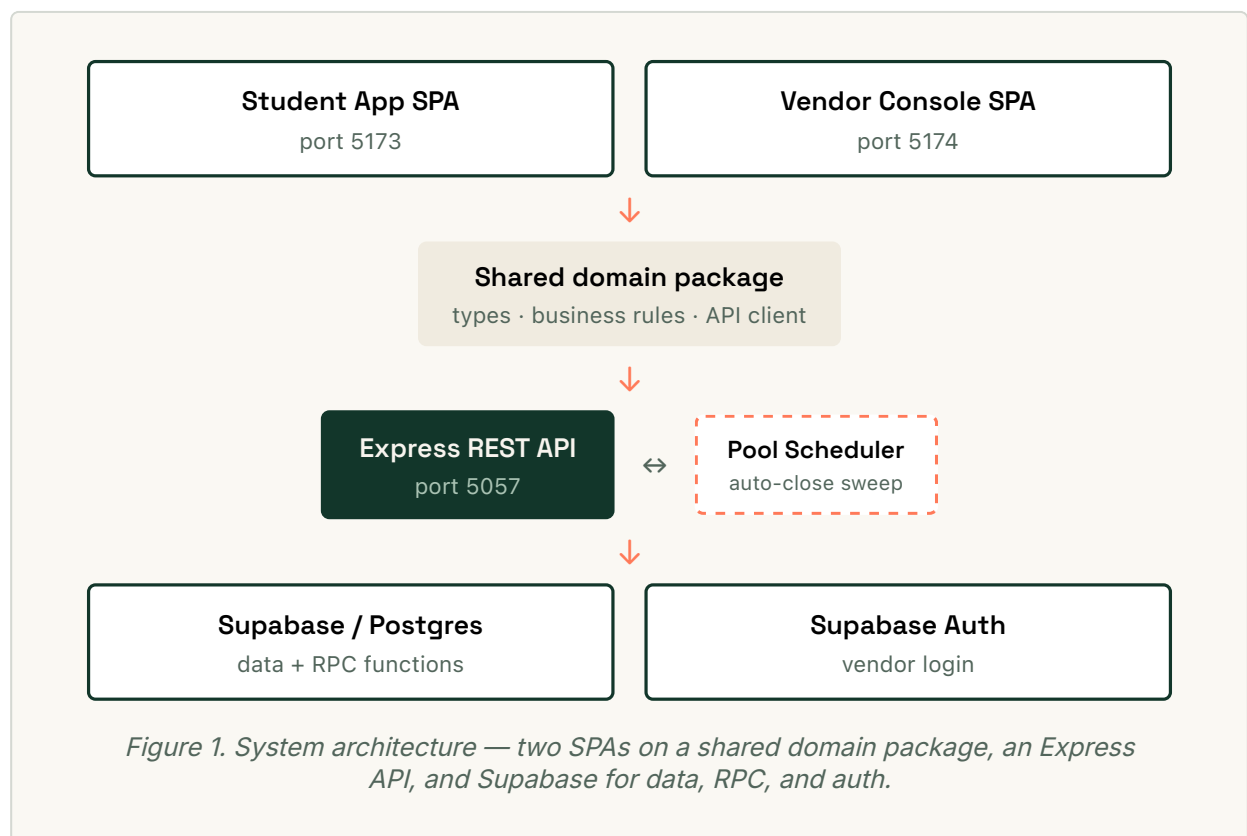
4.2 Non-Functional Requirements

- **Consistency:** the frontend polls the API every 5 seconds so vendor actions reflect on the student's tracking screen without a manual refresh.
- **Correctness under concurrency:** stock is decremented atomically inside a database transaction to prevent overselling when multiple students order simultaneously.
- **State integrity:** slot and order lifecycles are strict — invalid state transitions are rejected with HTTP 409.
- **Security:** vendor mutations require authentication; unauthenticated requests are rejected.
- **Low friction for students:** no account creation is required — identity is name-only.
- **Deployability:** the system runs on a serverless platform (Vercel); the pool-closing scheduler is implemented as a polling sweep compatible with serverless execution.

5. System Design

5.1 Architecture Overview

Two frontend single-page applications — the student app (port 5173) and the vendor console (port 5174) — both depend on a shared *domain* package that holds TypeScript types, business rules, and the API client. Both apps call an Express.js REST API (port 5057), which reads and writes Supabase/PostgreSQL, including Postgres RPC functions for atomic operations, and delegates vendor login to Supabase Auth. A background pool-scheduler process runs alongside the backend, polling for expired slots and auto-closing them via an internal sweep endpoint.



5.2 Data Flow

A student selects a hostel, which determines the active slot and vendor menu shown by the domain package's API client. Placing an order calls the API, which validates the slot is open, decrements inventory atomically, recomputes the slot's fee tier from its distinct order count, and returns the new state. The student app polls this state every 5 seconds to keep the

fee counter and order status current. A vendor closing a pool, dispatching it, or marking an order delivered triggers the same recomputation path, so the student view reflects vendor actions without a manual refresh.

5.3 Core Entities (Schema Overview)

The relational schema is organized around the following core entities and relationships:

Entity	Key Fields	Relationships
Hostel	id, name	has many Slots
Vendor	id, email, auth_id	has many MenuItems
MenuItem	id, vendor_id, name, price, stock	belongs to Vendor; referenced by OrderItems
Slot (Pool)	id, hostel_id, status, opens_at, closes_at	belongs to Hostel; has many Orders
Order	id, slot_id, student_name, status, fee	belongs to Slot; has many OrderItems
OrderItem	id, order_id, menu_item_id, quantity	belongs to Order and MenuItem
RestockLog	id, menu_item_id, amount, vendor_id, timestamp	belongs to MenuItem; audits vendor restocks

Table 1. Core entities inferred from the product's business rules; exact column names may differ in the checked-in schema.

5.4 API Overview

Representative REST endpoints exposed by the Express API:

Endpoint	Purpose
GET /hostels/:id/menu	Vendor menu for a hostel
GET /hostels/:id/slot	Current slot status and live fee
POST /orders	Place an order into the open slot
POST /vendor/login	Vendor authentication via Supabase Auth
GET /vendor/pools	All hostels' pools for the dashboard
POST /vendor/pools/:id/close	Close a pool early
POST /vendor/pools/:id/dispatch	Dispatch a closed pool
POST /vendor/orders/:id/deliver	Mark an order delivered
POST /vendor/inventory/:id/restock	Restock an item (preset or custom amount)
GET /vendor/analytics	Revenue, orders-by-hour, heatmap, hostel breakdown
POST /internal/sweep	Scheduler trigger: auto-close expired slots

6. SDLC Methodology

Poolit was built using an **iterative / incremental, Agile-influenced** SDLC rather than a strict waterfall. This fit the project because several key decisions — the fee-ladder thresholds, the exact student UX, and the right backend data store — could not be settled correctly on paper; they needed a working increment to test against. Each phase of the project delivered something runnable: a frontend prototype to validate UX before any backend existed, a first backend to validate the API shape, a database migration once relational needs became clear, and a hardening pass once real usage surfaced concurrency issues. The classic SDLC stages (Requirements, Design, Implementation, Testing, Deployment, Maintenance) were therefore visited multiple times, not once.



Figure 2. Standard SDLC stages, revisited across ten development phases.

SDLC Stage	Project Phase(s)
Requirements	1 — Requirements gathering & problem definition
Design	2 — Frontend prototype; 6 — Vendor console UI revamp
Implementation	2–5 — Prototype through vendor authentication; 7 — Deployment
Testing	9 — Testing & QA
Deployment	7 — Deployment to Vercel
Maintenance	8 — Hardening pass; 10 — Final report & demo prep

7. Project Timeline

The ten development phases were run as roughly one- to two-week sprints across a 15-week semester, starting August 10, 2026. As of this report (September 16, 2026), phases 1–4 are complete and phase 5 (vendor authentication) is in progress.

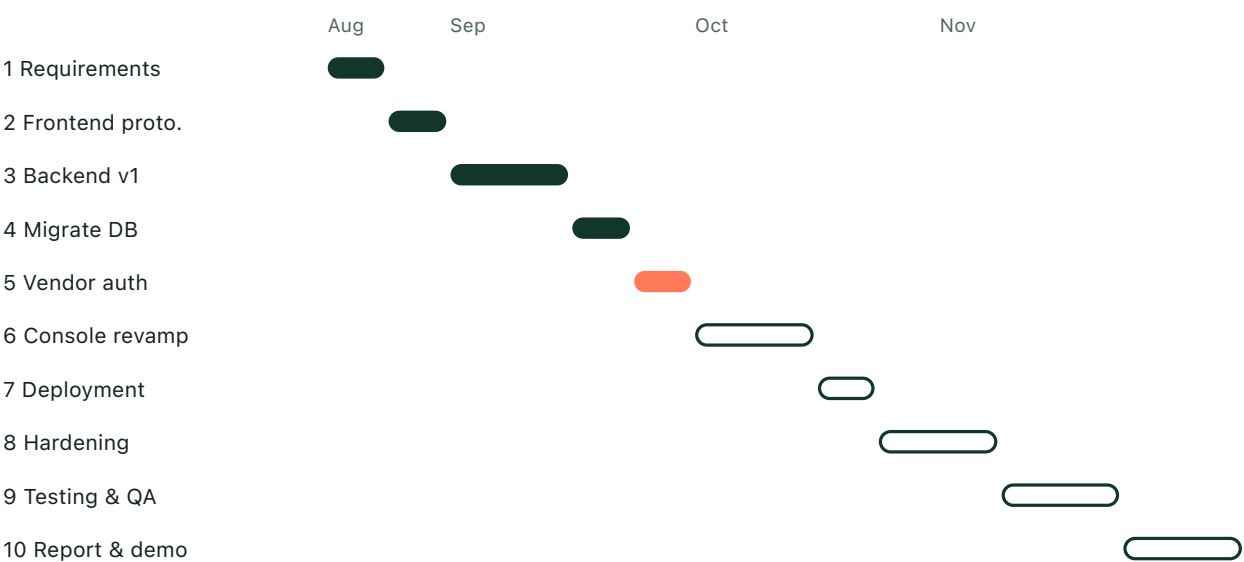


Figure 3. Gantt chart, weeks 1–15, Aug 10–Nov 22, 2026. Dark = completed, coral = in progress, outlined = planned.

Phase	Weeks	Dates	Status
1. Requirements & problem definition	1	Aug 10–16	Completed
2. Frontend prototype	2	Aug 17–23	Completed
3. Backend v1 (MongoDB)	3–4	Aug 24–Sep 6	Completed
4. Migration to Supabase/Postgres	5	Sep 7–13	Completed
5. Vendor authentication	6	Sep 14–20	In progress
6. Vendor console UI revamp	7–8	Sep 21–Oct 4	Planned
7. Deployment	9	Oct 5–11	Planned
8. Hardening pass	10–11	Oct 12–25	Planned
9. Testing & QA	12–13	Oct 26–Nov 8	Planned
10. Final report, presentation & demo prep	14–15	Nov 9–22	Planned

8. Implementation Details

8.1 Frontend

Both the student app and vendor console are React + TypeScript SPAs built with Vite, organized as a monorepo. A shared *domain* package holds types, business rules (e.g., fee-ladder computation), and the API client, so both apps stay consistent with the same source of truth instead of duplicating logic. The frontend prototype phase built both apps against an in-memory reducer with localStorage persistence before any backend existed, which let the team validate UX and business rules early and produce a backend handoff specification.

8.2 Backend

The backend is an Express.js REST API. It was first built on MongoDB, implementing the handoff specification from the prototype phase, and later migrated to Supabase/PostgreSQL for relational integrity and built-in auth and RPC support. Postgres RPC functions perform atomic operations — most importantly, decrementing inventory and recomputing the slot fee within a single transaction on order placement.

8.3 Database & Auth

Supabase provides both the PostgreSQL database and authentication. Vendor authentication uses Supabase Auth (email + password); the backend rejects unauthenticated vendor mutations. Students remain frictionless and unauthenticated, identified by name only, which keeps the ordering flow fast for a low-stakes, trusted hostel context.

8.4 Deployment

The system is deployed to Vercel. The pool-closing scheduler runs as a polling-based sweep rather than a long-lived background process, so it remains compatible with Vercel's serverless execution model, triggered via an internal sweep endpoint.

9. Business Logic Details

9.1 Fee Ladder

The delivery fee is a function of the number of distinct orders in a slot — not item quantity or order value — recomputed live while the slot is open, and locked in once the slot closes:

Orders in Slot	Fee
1–2	₹20
3–5	₹10
6–9	₹5
10+	Free

9.2 Slot & Order Lifecycles

Slot lifecycle: Open → Closed → Dispatched, transitioning via timer expiry or manual vendor action.

Order lifecycle: Placed → Pooled → Dispatched → Delivered — strict, with no skipped states; invalid transitions are rejected with HTTP 409.

9.3 Inventory Rules

Stock is decremented atomically inside a database transaction on order placement, preventing two concurrent orders from overselling the same item. Restocking — via a preset amount or a custom amount — is an audited vendor action, and low-stock alerts surface in the console before an item runs out.

10. Testing Approach

Testing focused on the areas most likely to break under real use: state-transition correctness and concurrency. Slot and order lifecycle transitions were exercised to confirm that invalid transitions are rejected with HTTP 409 rather than silently accepted. Inventory operations were tested under concurrent order placement to confirm that atomic stock decrements prevent overselling. The fee ladder was verified against its boundary counts (1–2, 3–5, 6–9, 10+ orders) to confirm the correct tier is always shown. The 5-second polling sync was checked end-

to-end — a vendor closing a pool or marking an order delivered was confirmed to reach the student's tracking screen without a manual refresh. This testing was consolidated into a dedicated QA phase (phase 9) ahead of the final demo.

11. Results & Outcomes

The project delivered a working, deployed system: a student app and a vendor console, both live on Vercel, backed by a Supabase/PostgreSQL database and an authenticated Express API. The core fee-ladder mechanic and the slot/order lifecycles function end to end, and the hardening pass resolved the concurrency and query-performance issues (race conditions, N+1 queries) found once the system was exercised more heavily, along with introducing a CORS allowlist and internal documentation. The iterative approach — prototype first, then backend, then migration, then authentication, then polish — meant that each stage of the project had a working system to build on and test against.

12. Limitations & Future Scope

12.1 Known Limitations

- No real product photography yet — menu items use emoji-based placeholder tiles.
- No per-store vendor account scoping — every vendor currently sees every hostel/store.
- Delivery riders are static/simulated; there is no real courier integration.

12.2 Planned Next

- Checked-in database schema and migrations.
- Local JWT verification.
- Per-store vendor scoping.
- Pagination on list endpoints.
- Real product imagery and real dispatch integration.

13. Conclusion

Poolit shows how an iterative SDLC can take a hostel logistics problem from an in-memory prototype to a deployed, authenticated, Supabase-backed product. Its core contribution is the live-dropping fee counter: a simple, visible incentive that turns individual hostel orders into a coordinated pool, cutting delivery cost per student and delivery trips per vendor. The remaining gaps — per-store vendor scoping, real courier integration, and real product imagery — are well-scoped next steps rather than open design questions.

14. References

- React — react.dev
- TypeScript — typescriptlang.org
- Vite — vitejs.dev
- Express.js — expressjs.com
- Supabase Documentation — supabase.com/docs
- PostgreSQL Documentation — postgresql.org/docs
- Vercel Documentation — vercel.com/docs